Results

Results I

This section reports repository facts produced by the meta-project's live introspection pass. Test counts, module inventories, pipeline-stage counts, and figure data are injected from generated artifacts. Runtime and achieved-coverage claims are intentionally omitted when no versioned benchmark or coverage artifact backs them.

Multi-Project Pipeline Execution I

The `./run.sh` interactive orchestrator can execute the public roster through the shared DAG. The table below is a structural snapshot, not a timing benchmark: counts come from repository introspection, and the coverage column states each exemplar's declared policy floor rather than an unversioned observed percentage.

Multi-Project Pipeline Execution I

Project	Effective core stages[1]	Discovered tests	Declared project floor
`template_code_project`	8	258	90%
`template_prose_project`	8	165	90%
`template_autoresearch_project`	8	318	90%

Multi-Project Pipeline Execution I

[1] "Core-only" excludes LLM-tagged and other opt-in stages according to the YAML stage tags. A fresh run must be used to establish completion status or wall-clock performance for a particular machine and dependency set.

Infrastructure Test Suite I

Metric	Value
Test files	997+
Total tests	~13,093
Infrastructure coverage gate	≥60% configured floor
Prohibited mock-framework imports	Checked by the static no-mocks gate

Infrastructure Test Suite I

Exercises Pandoc/XeLaTeX paths, steganography hashing, telemetry, YAML-driven pipeline DAG resolution, HTTPS-bound optional suites (`pytest-httpserver`), and local Ollama-gated subsets.

Infrastructure Module Inventory I

The introspection module (`template_template.core.introspection`) emits the authoritative table below—every row reflects `discover_infrastructure_modules(REPO_ROOT)`.

Infrastructure Module Inventory I

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
`autoresearch`	11	✓	✓	`build_autoresearch_plan`, readiness validation CLI
`benchmark`	4	✓	✓	Template harness scoring + comparative gates
`config`	0	✓	✓	Repository defaults + hardened templates
`core`	202	✓	✓	`get_logger`, `load_config`, `TemplateError`
`docker`	0	✓	✓	Containerisation scaffolding
`doctor`	14	✓	✓	Checkout diagnose/fix/undo repairs
`documentation`	23	✓	✓	`FigureManager`, `build_api_index`, `generate_markdown_table`
`fonds`	6	✓	✓	—

Infrastructure Module Inventory I

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
`formal_contracts`	5	✓	✓	—
`llm`	57	✓	✓	Ollama helpers, sanitization, review + translation pipelines
`logrotate.d`	0	✓	✓	Rotation snippets (documentation-first)
`metadata`	3	✓	✓	`normalized_repository_url`, `generate_metadata_package`
`methods`	10	✓	✓	`build_methods_orchestration_plan`, methods-stage contracts + validation
`orchestration`	14	✓	✓	`PipelineRunner`, entry point for `./run.sh`
`project`	123	✓	✓	`discover_projects`, workspace management
`prose`	9	✓	✓	Markdown readability + prose tooling

Infrastructure Module Inventory I

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
`provenance`	9	✓	✓	—
`publishing`	89	✓	✓	Zenodo, executable bundle, archival targets
`reference`	16	✓	✓	BibTeX models, parsers, converters
`rendering`	123	✓	✓	PDF/HTML/slide rendering, Pandoc filters
`reporting`	416	✓	✓	Coverage parsers, dashboards, executive artefacts
`research`	3	✓	✓	—
`rules`	6	✓	✓	—
`scientific`	4	✓	✓	`check_numerical_stability`, `benchmark_function`

Infrastructure Module Inventory I

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
`search`	77	✓	✓	`infrastructure.search.literature` clients + cache
`sia`	10	✓	✓	Self-Improving-AI loop: task validation, harness, metric capture
`skills`	8	✓	✓	`discover_skills`, SKILL manifest regeneration
`steganography`	14	✓	✓	Watermark overlays + hash manifests
`tools`	6	✓	✓	—
`transmission`	6	✓	✓	—
`validation`	87	✓	✓	PDF + Markdown + integrity CLIs

Infrastructure Module Inventory I

All 31 enumerated subdirectories carry Tier-1/`README.md` and Tier-2/`AGENTS.md` coverage wherever the Documentation Duality standard applies; subsets ship Tier-3 `SKILL.md` descriptors for MCP routing (`infrastructure/skills` manifest generation).

Agentic Skill Documentation Coverage I

Layer	Role
Shared agent guidance	Root `AGENTS.md`, `.agents/` skills, roles, and workflows
Structural	`AGENTS.md` directories
Skills	Optional `SKILL.md` manifests + generated `.agents/skill_manifest.json`
PAI capsule	Repository level `PAI.md` narratives

Agentic Skill Documentation Coverage I

385+ Markdown shards under `docs/` capture operational knowledge without duplicating auto-generated inventories.

DAG Reference (Declarative Executor) I

The table summarizes the 10 default full-run stages (core plus live LLM nodes) from 17 YAML declarations; core mode selects 8. Separately tagged opt-in extensions are omitted here. Entries follow dependency order rather than numeric script filenames.

DAG Reference (Declarative Executor) I

Name	Typical script / method	Responsibility	Failure semantics
Clean Output Directories	`_run_clean_outputs`	Deletes stale output/ trees	Blocking
Environment Setup	`scripts/pipeline/stage_00_setup.py`	Validates tooling, PYTHONPATH scaffolding	Blocking
Infrastructure Tests	`scripts/pipeline/stage_01_test.py --infra-only --infra-scope pipeline-smoke`	Focused infrastructure smoke contracts	Required tests must pass
Project Tests	`scripts/pipeline/stage_01_test.py --project-only`	Project-local tests + coverage gates	Required tests and coverage must pass
Project Analysis	`scripts/pipeline/stage_02_analysis.py`	Executes `projects/<name>/scripts/*.py`	Blocking
PDF Rendering	`scripts/pipeline/stage_03_render.py`	Pandoc → XeLaTeX manuscripts	Blocking
Output Validation	`scripts/pipeline/stage_04_validate.py`	Structural PDF/markdown probes	Blocking / warnings
LLM Manuscript Review	`scripts/pipeline/stage_06_llm_review.py --reviews-only`	Local Ollama reviews	Fresh policy-bound skip; required live acceptance remains blocking

DAG Reference (Declarative Executor) I

Name	Typical script / method	Responsibility	Failure semantics
LLM Translations	`scripts/pipeline/stage_06_llm_review.py --translations-only`	Optional translations	Fresh policy-bound skip
Copy Outputs	`scripts/pipeline/stage_05_copy.py`	Mirrors deliverables → `output/<project>/`	Blocking

DAG Reference (Declarative Executor) I

`scripts/pipeline/stage_07_executive_report.py` provides separate **multi-project executive reporting**, not an additional single-project DAG node (`execute_pipeline.py`). The multi-project executor writes its aggregate summary after project execution.

Steganographic Performance Boundary I

The steganography path exposes metadata injection, SHA-256 hashing, overlay generation, and optional QR-code insertion as separately testable operations. This revision makes no latency claim because the project does not currently generate a versioned benchmark artifact recording hardware, input PDF, warm-up policy, repetitions, and dispersion. A future performance table should be compiled from such an artifact rather than copied from an ad-hoc workstation run.

Self-Referential Analysis I

Rendered via `projects/templates/template_template` (`generate_manuscript_metrics.py` → injected tokens such as 31). Architecture figures stem from `template_template.figures.architecture_viz`—font sizes constrained by §QA.

Self-Referential Analysis I

Infrastructure · 1 / 8

autoresearch 11 py [ARS·]	**benchmark** 4 py [ARS·]
config 0 py [ARS·]	**core** 204 py [ARS·]

Self-Referential Analysis I

Infrastructure · 2 / 8

docker 0 py [ARS·]	**doctor** 14 py [ARS·]
documentation 23 py [ARS·]	**fonds** 6 py [ARS·]

Self-Referential Analysis I

Infrastructure · 3 / 8

formal contracts 5 py [ARS·]	**llm** 57 py [ARS·]
logrotate.d 0 py [AR··]	**metadata** 3 py [ARS·]

Self-Referential Analysis I

Infrastructure · 4 / 8

methods
10 py
[ARS·]

orchestration
14 py
[ARS·]

project
123 py
[ARS·]

prose
9 py
[ARS·]

Infrastructure · 5 / 8

provenance

9 py
[ARS·]

publishing

89 py
[ARS·]

reference

16 py
[ARS·]

rendering

125 py
[ARS·]

Infrastructure · 6 / 8

reporting

418 py
[ARS·]

research

3 py
[ARS·]

rules

6 py
[ARS·]

scientific

4 py
[ARS·]

Self-Referential Analysis I

Infrastructure · 7 / 8

search

77 py
[ARS·]

sia

10 py
[ARS·]

skills

8 py
[ARS·]

steganography

15 py
[ARS·]

Self-Referential Analysis I

Infrastructure · 8 / 8

tools

6 py [ARS·]

transmission

6 py [ARS·]

validation

87 py [ARS·]

Self-Referential Analysis I

Projects · 1 / 7

active inference

16 chapters
84 test files

advanced literature review

23 chapters
8 test files

autopoiesis

8 chapters
31 test files

autore search project

6 chapters
23 test files

Self-Referential Analysis I

Projects · 2 / 7

autoscientists

9 chapters
14 test files

code project

9 chapters
16 test files

data descriptor

8 chapters
7 test files

eda notebook

9 chapters
12 test files

Projects · 3 / 7

experiment tree

6 chapters
5 test files

formal

7 chapters
34 test files

gold refinement

12 chapters
25 test files

literature meta analysis

21 chapters
74 test files

Projects · 4 / 7

madlib

12 chapters
8 test files

methods paper

9 chapters
12 test files

newspaper

6 chapters
10 test files

pitch deck

7 chapters
14 test files

Projects · 5 / 7

pools rules tools

8 chapters
13 test files

prose project

8 chapters
13 test files

redacted report

3 chapters
7 test files

registered report

8 chapters
5 test files

Self-Referential Analysis I

Projects · 6 / 7

search project

9 chapters
33 test files

sia

6 chapters
16 test files

storybook

6 chapters
7 test files

template

21 chapters
11 test files

Projects · 7 / 7

textbook

51 chapters
13 test files

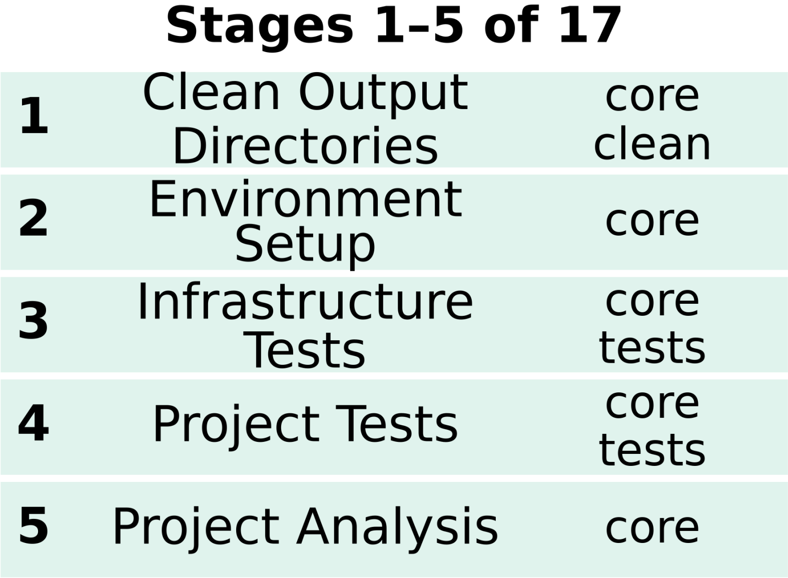
Self-Referential Analysis I

Figure 3: Live rendering of the Two-Layer Architecture from repository introspection: the infrastructure layer (top) holds the `31` enumerated infrastructure directories, each annotated with its Python file count and a four-slot documentation badge—`A` AGENTS.md, `R` README.md, `S` SKILL.md, `P` PAI.md, with · marking an absent file—so a fully documented module reads `[ARSP]`. A YAML DAG arrow connects it to the project

Self-Referential Analysis I

layer (bottom) of public exemplars labelled with chapter and test-file counts. The takeaway: documentation-duality coverage is near-uniform across the infrastructure, and every box was placed from the same live data the prose cites.

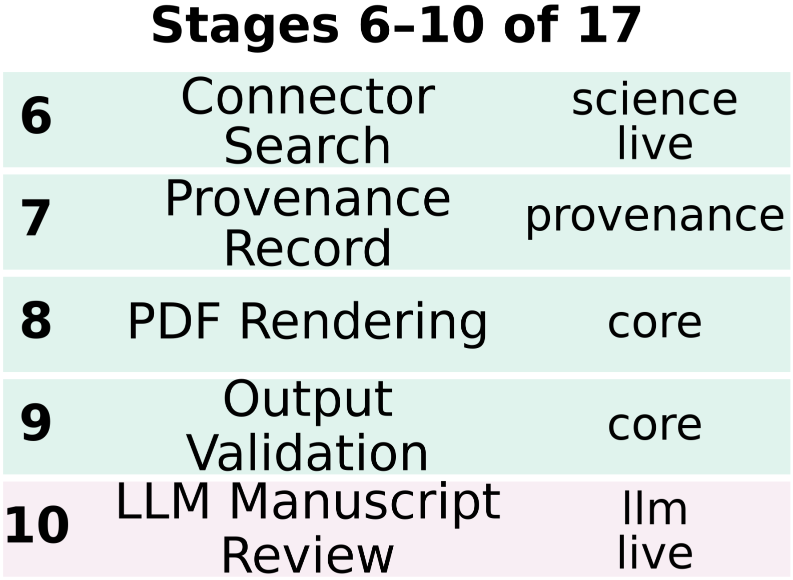
Self-Referential Analysis I



Stages 1–5 of 17

1	Clean Output Directories	core clean
2	Environment Setup	core
3	Infrastructure Tests	core tests
4	Project Tests	core tests
5	Project Analysis	core

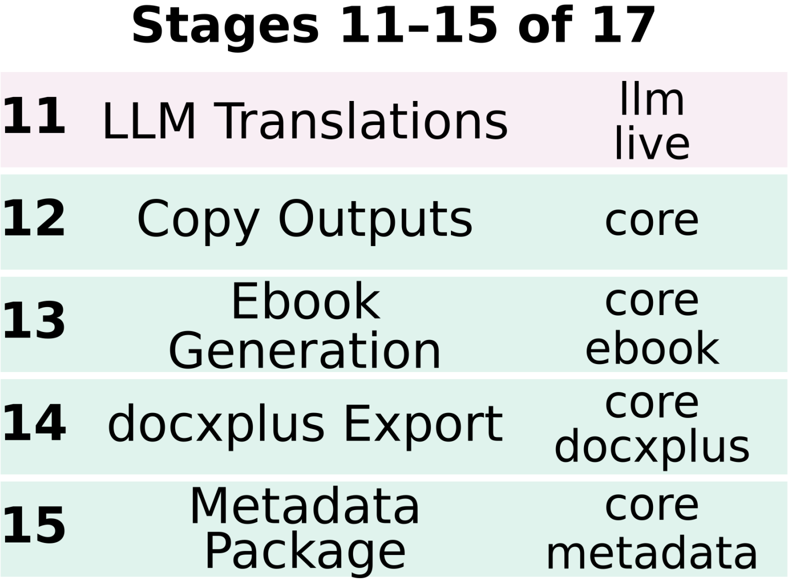
Self-Referential Analysis I



Stages 6–10 of 17

6	Connector Search	science live
7	Provenance Record	provenance
8	PDF Rendering	core
9	Output Validation	core
10	LLM Manuscript Review	llm live

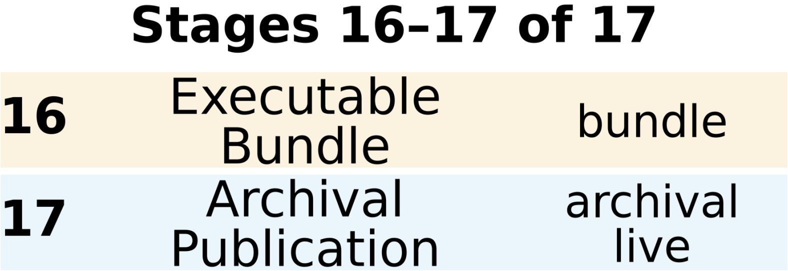
Self-Referential Analysis I



Stages 11–15 of 17

11	LLM Translations	llm live
12	Copy Outputs	core
13	Ebook Generation	core ebook
14	docxplus Export	core docxplus
15	Metadata Package	core metadata

Self-Referential Analysis I



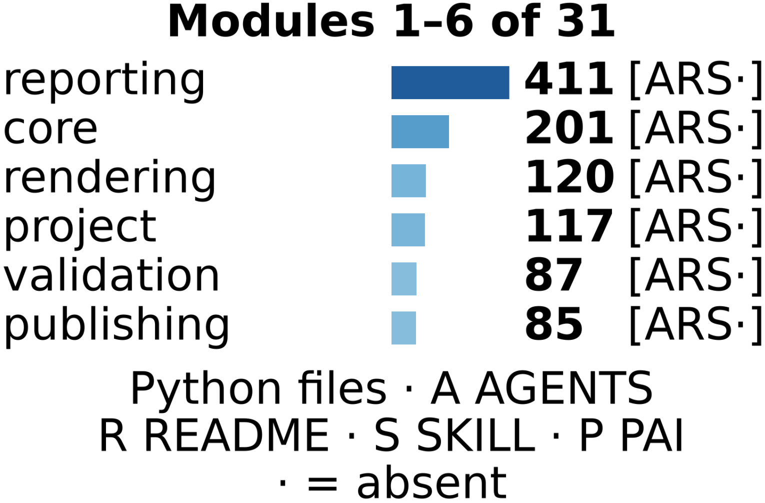
Stages 16–17 of 17

16	Executable Bundle	bundle
17	Archival Publication	archival live

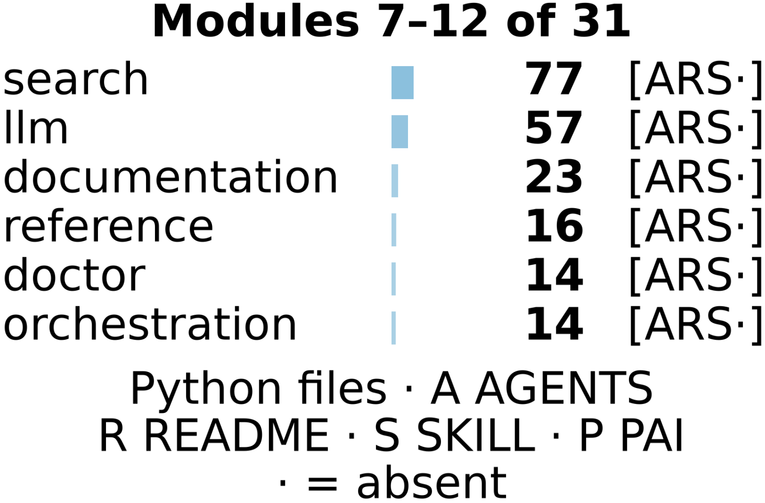
Self-Referential Analysis I

Figure 4: Declared-stage reference with all 17 YAML stages in reading order, showing complete names and tags (core, LLM, ebook, docxplus, metadata, bundle, archival). Panel order documents declarations; it does not imply a linear dependency chain.

Self-Referential Analysis I



Self-Referential Analysis I



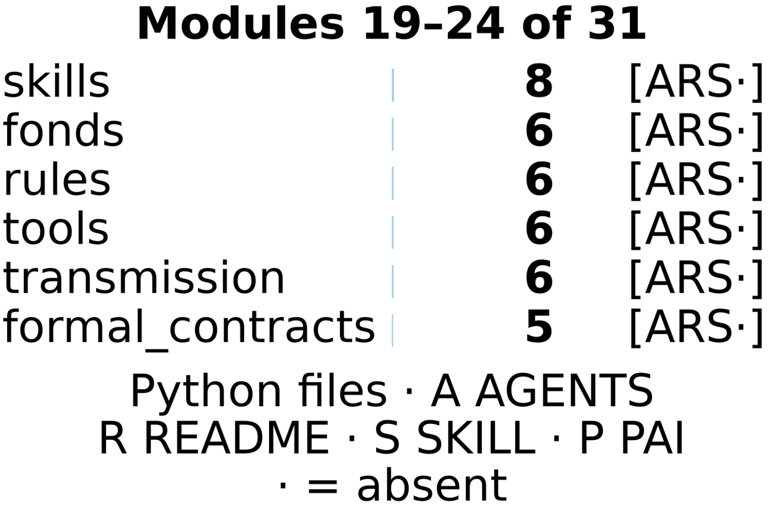
Self-Referential Analysis I

Modules 13–18 of 31

steganography	**13**	[ARS·]
autoresearch	**11**	[ARS·]
methods	**10**	[ARS·]
sia	**10**	[ARS·]
prose	**9**	[ARS·]
provenance	**9**	[ARS·]

Python files · A AGENTS
R README · S SKILL · P PAI
· = absent

Self-Referential Analysis I

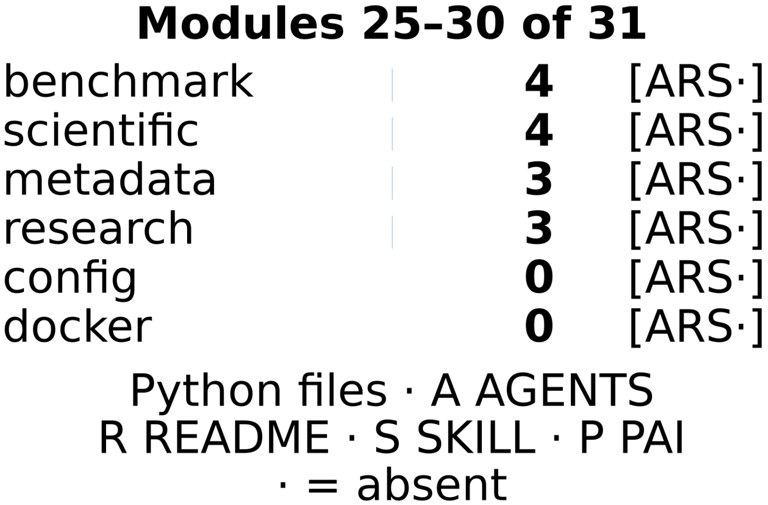


Modules 19–24 of 31

skills	**8**	[ARS·]
fonds	**6**	[ARS·]
rules	**6**	[ARS·]
tools	**6**	[ARS·]
transmission	**6**	[ARS·]
formal_contracts	**5**	[ARS·]

Python files · A AGENTS
R README · S SKILL · P PAI
· = absent

Self-Referential Analysis I



Modules 25–30 of 31

benchmark	**4**	[ARS·]
scientific	**4**	[ARS·]
metadata	**3**	[ARS·]
research	**3**	[ARS·]
config	**0**	[ARS·]
docker	**0**	[ARS·]

Python files · A AGENTS
R README · S SKILL · P PAI
· = absent

Modules 31–31 of 31

logrotate.d	**0**	[AR··]

Python files · A AGENTS
R README · S SKILL · P PAI
· = absent

Self-Referential Analysis I

Figure 5: Horizontal file-count histogram of every infrastructure subdirectory, sorted largest-first.

Self-Referential Analysis I

The long tail of small, single-purpose packages beside a handful of larger ones (`core`, `validation`, `publishing`) is the visual signature of the Unix-philosophy modularity the architecture section argues for—capability concentrated where it compounds, not spread evenly by fiat.

Comparative Feature Analysis I

The comparative feature matrix below summarizes the Appendix F capability matrix.

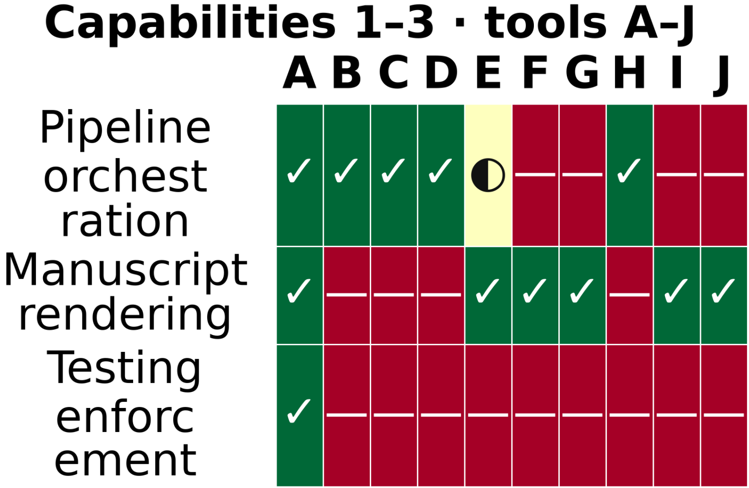
Comparative Feature Analysis I

Tool key · A–J

A template/
B Snakemake 9.x
C Nextflow 25.x
D CWL 1.2
E Quarto 1.x
F Jupyter Book 2.x
G R Markdown
H DVC 3.x
I Overleaf (2025)
J OpenAI Prism

✓ full · ◐ partial · — absent

Comparative Feature Analysis I

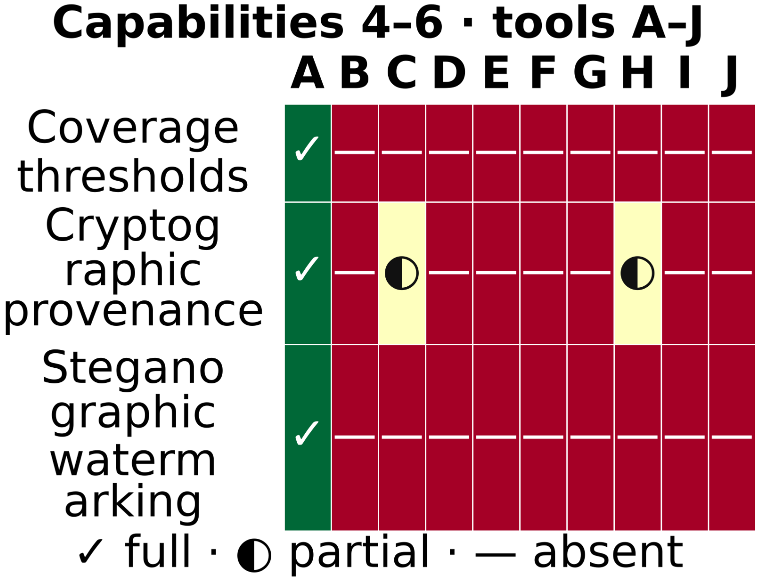


Capabilities 1–3 · tools A–J

	A	B	C	D	E	F	G	H	I	J
Pipeline orchestration	✓	✓	✓	✓	◐	—	—	✓	—	—
Manuscript rendering	✓	—	—	—	✓	✓	✓	—	✓	✓
Testing enforcement	✓	—	—	—	—	—	—	—	—	—

✓ full · ◐ partial · — absent

Comparative Feature Analysis I



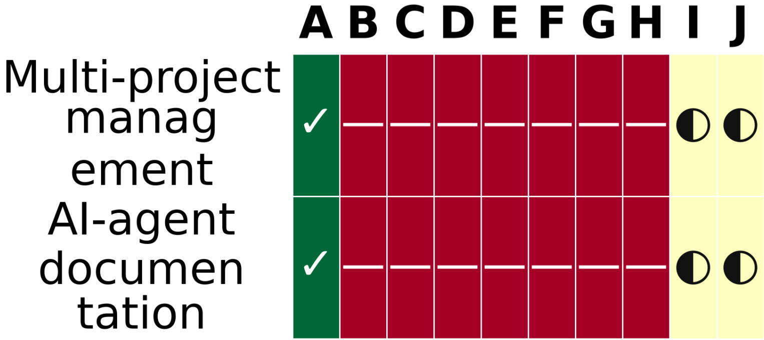
Capabilities 4–6 · tools A–J

	A	B	C	D	E	F	G	H	I	J
Coverage thresholds	✓	—	—	—	—	—	—	—	—	—
Cryptographic provenance	✓	—	◐	—	—	—	—	◐	—	—
Steganographic watermarking	✓	—	—	—	—	—	—	—	—	—

✓ full · ◐ partial · — absent

Comparative Feature Analysis I

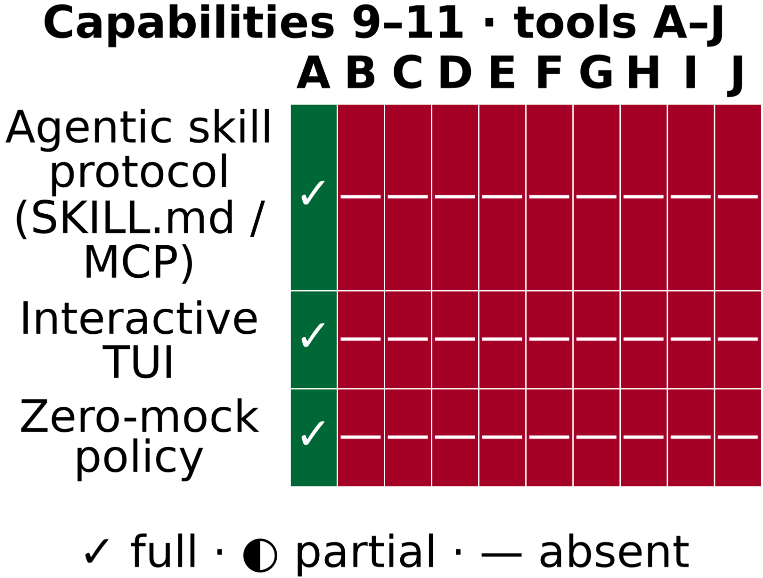
Capabilities 7–8 · tools A–J



	A	B	C	D	E	F	G	H	I	J
Multi-project management	✓	—	—	—	—	—	—	—	◐	◐
AI-agent documentation	✓	—	—	—	—	—	—	—	◐	◐

✓ full · ◐ partial · — absent

Comparative Feature Analysis I

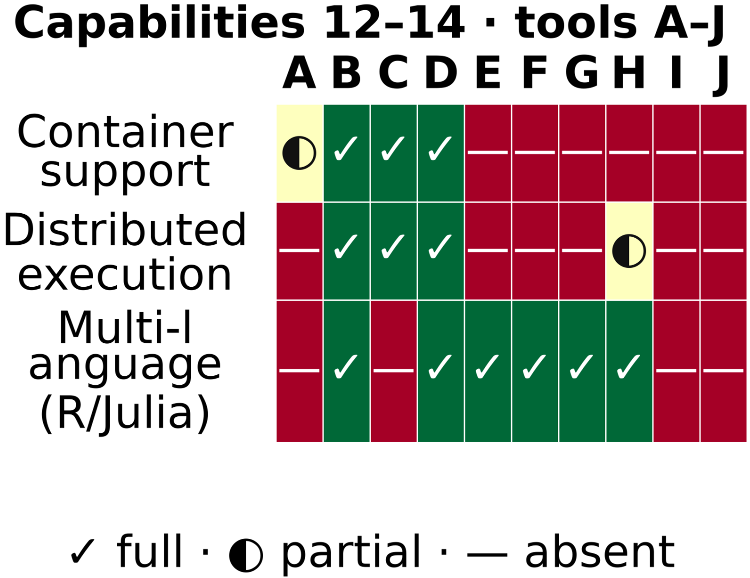


Capabilities 9–11 · tools A–J

	A	B	C	D	E	F	G	H	I	J
Agentic skill protocol (SKILL.md / MCP)	✓	—	—	—	—	—	—	—	—	—
Interactive TUI	✓	—	—	—	—	—	—	—	—	—
Zero-mock policy	✓	—	—	—	—	—	—	—	—	—

✓ full · ◐ partial · — absent

Comparative Feature Analysis I



Capabilities 12–14 · tools A–J

	A	B	C	D	E	F	G	H	I	J
Container support	◐	✓	✓	✓	—	—	—	—	—	—
Distributed execution	—	✓	✓	✓	—	—	—	◐	—	—
Multi-language (R/Julia)	—	✓	—	✓	✓	✓	✓	✓	—	—

✓ full · ◐ partial · — absent

Comparative Feature Analysis I

Figure 6: 14×10 heatmap annotated in appendix text—green ✓ full native capability, yellow partial / plugin-hosted, red — unavailable. Rows group *Core Pipeline*, *Quality & Security*, then *Ecosystem*.

Comparative Feature Analysis I

[1] Nextflow lineage records exist at workflow scope, not per rendered PDF citation graph.

[2] DVC: content-addressed artifacts without native prose rendering.

[3] DVC: remote object stores (S3, GCS, Azure) without turnkey CI manuscript gates.

Test Quality Metrics I

- **Zero mocks:** repository policy bans `unittest.mock` / patching frameworks in tests.
- **Filesystem + subprocess realism:** ephemeral directories + actual CLI binaries.
- **HTTP realism:** infra suites favour `pytest-httpserver`; literature tests hit recorded fixtures.
- **`template_code_project`** focuses on numerical reproducibility assertions.
- **`template_autoresearch_project`** exercises the AutoResearch readiness planner (`infrastructure/autoresearch/`). **`template_search_project`** exercises literature-search workflows.